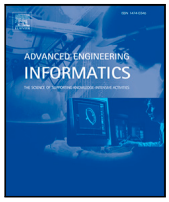
Advanced Engineering Informatics 62 (2024) 102735

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei



# Intermediate representations to improve the semantic parsing of building regulations

Stefan Fuchs [a,*], Johannes Dimyadi [a,b], Michael Witbrock [a], Robert Amor [a]

[a] School of Computer Science, The University of Auckland, 34 Princes Street, Auckland, 1010, New Zealand
[b] CAS (Codify Asset Solutions Limited), Auckland, New Zealand

ARTICLE INFO

Dataset link: https://github.com/stefan-1992/intermediate-representations

*Keywords:*
Semantic parsing
Building regulation
Transformer
Intermediate representation
Automated compliance checking

ABSTRACT

Recent developments show that large transformer-based language models have the capability to generate coherent text and source code in response to user prompts. This capability can be used in the construction domain to interpret building regulations and convert them into a formal representation usable for automated compliance checking. While base-size models can already be taught to perform semantic parsing with decent quality, this paper shows how Intermediate Representations (IRs) can be used to improve the semantic parsing quality. With reversible IRs, the training time was reduced to almost a quarter of the initial duration, and through adding a hierarchical parsing step, improvements of up to 6.6% on F1 scores were reached. Furthermore, intermediate representations provide a novel and interpretable method towards a human-in-the-loop approach for translating building regulations into a formal representation.

## 1. Introduction

Semantic parsing has many applications, from program synthesis to interpreting robotic commands and querying in natural language. Another potential use case is the interpretation of norms such as regulations and standards, which is essential for Automated Compliance Checking (ACC) applications. ACC is an active area of research, particularly in the Architecture, Engineering & Construction (AEC) domains. Checking a building design for compliance was conceptualised by Eastman et al. [1] as a four-step process, which can be summarised as:

1. The building regulations and standards need to be interpreted and made understandable for a computer.
2. The building design information needs to be extracted from drawings, documentation, or the Building Information Model (BIM). It needs to be ensured that the information requirements are met, and additional information might need to be inferred.
3. Both information streams need to be aligned or directly consumed by a reasoner. The reasoning capabilities start from simple quantitative and existential checks but can also involve complex calculations, models, and simulations.
4. The compliance results then need to be reported. In the case of non-compliant instances, the corresponding normative requirements, as well as the exact location and involved building elements, should be reported.

The primary challenges in realising these steps are the absence or lack of digital regulations to convey the highly complex legalese, the availability and accessibility of information from BIM, the diversity of required checking procedures (e.g., spatial reasoning), and non-conformity of terms used in BIM and regulations [2].

A common approach to address the machine-understandability of natural language regulations is translating them into a computable form, which an ACC system can then utilise. Both the translation process and various formats' representation capabilities have received significant attention [3–5]. Approaches to translating building regulations range from manual expert interpretation to fully automated translation. While most previous approaches divided the translation process into one or more information extraction (e.g., entities, properties, relations, exceptions) and transformation tasks, Fuchs et al. [6] proposed to tackle the problem as a seq2seq task using a transformer-based semantic parser. A seq2seq model is trained with pairs of input and target sequences, allowing the model to learn the mapping between the two. It is commonly used in tasks such as language translation, summarisation, and question-answering.

While Fuchs et al. [7] showed that semantic parsing is a promising direction, the performance of existing semantic parsers for building regulations still falls short of expectations. This paper investigates Intermediate Representations (IRs) for a faster and more accurate translation of building regulations into LegalRuleML (LRML), an emerging eXtensible

* Corresponding author.
*E-mail address:* sffc348@aucklanduni.ac.nz (S. Fuchs).

https://doi.org/10.1016/j.aei.2024.102735
Received 1 April 2024; Received in revised form 20 June 2024; Accepted 19 July 2024
Available online 2 August 2024

Markup Language (XML)-based representation standard [8], unlocking additional possibilities for interpretability and semi-automation. IRs are simplified representations that can either be reverted deterministically to the original representation or used as an intermediate step for hierarchical semantic parsing. The suitability of various IRs and whether multi-task learning or training separate specialised models performs better for semantic parsing of building regulations is examined. This paper contributes a method for a faster and more accurate translation of building regulations into LRML and unlocks new possibilities for interpretability and semi-automation using IRs. The work published in Fuchs et al. [9] is extended by providing additional details on the methodology, further analysis of the results, and an extended discussion analysing limitations.

## 2. Background

### 2.1. Automated compliance checking

ACC refers to the process of automatically evaluating whether someone or something conforms with the applicable legal requirements. In the context of this paper, this is the conformance with legal requirements concerning the construction and design of buildings. The rising adoption of BIM and, with it, the availability of the building design in a structured format has been a significant contributor to ACC.

Several tools have successfully checked BIM for compliance issues. A recent literature review by Amor and Dimyadi [10] provides an overview of ACC tools, the timeframe of their application, and open challenges for the broad adoption of ACC. Many other reviews can be found on this topic (e.g., Dimyadi and Amor [11], Ismail et al. [12], Beach et al. [13], and Aydın [14]).

Solibri Model Checker[1] is one of the most renowned commercial tools on the market. It allows one to check a BIM against a range of predefined rule sets, as well as newly parametrised rules. The building information can be easily accessed, and a broad range of functions for numeric and spatial comparisons are available. However, the weaknesses of this tool are the limited number of rule sets and having these rule sets in a vendor-specific format. Although the user can parameterise and add rules, doing so manually for all applicable provisions is very labour-intensive. Furthermore, one is limited to the checking functionality of the rule engine.

More recently, UpCodes[2] started the endeavour of providing an integrated and fully searchable version of a range of building codes and other normative documents and developing a range of tools around it, such as an Artificial Intelligence (AI) assistant for question answering and a Revit plugin for compliance checking. There were also many government-funded or supported efforts in different countries towards ACC systems; examples are listed below.

- CORENET e-PlanCheck in Singapore can be described as a black box system, where rules were hard-coded into the software, but which can check complex requirements [15,16]. It is based on the FORNAX platform [17], which provides the ability to enrich Industry Foundation Classes (IFC) models and the functionality required for ACC and offers a level of abstraction between building requirements and design. Since 2018, Singapore has developed CORENET-X,[3] which launched at the end of 2023 and helps to streamline and automate parts of the compliance checking process based on IFC building models. In particular, they extended IFC to IFC-SG[4] through user-defined object types, property sets, and properties to capture the information required to check Singapore's building requirements.
- DesignCheck in Australia [18] is another example of an object-based approach storing the formalised requirements in the Express Data Manager (EDM). During the object-based interpretation of building regulations, relevant building objects, properties, and relations between them were determined. This included the relevant domain-specific knowledge required for checking specific requirements.
- SMARTcodes [19], in the United States, led by the International Code Council, was based on marked-up building codes. The markup format continued to be developed by AEC3 into Requirement, Applies, Select, and Exception (RASE) [20], and for example, was utilised by Beach et al. [21] for ACC.
- KBim, in Korea, formalised the Korean Building Act in KBimCode and used that formal representation for compliance checks [22]. They experimented with Natural Language Processing (NLP) techniques to support the formalisation [23].
- ACABIM [24] developed by Compliance Audit Systems (CAS) in New Zealand follows a human-centred workflow-driven approach to fully automate quantitative checks and request user input for qualitative requirements.

Zhang and El-Gohary [25] introduced Semantic Natural language processing (NLP)-based Automated Compliance Checking (SNACC), the first compliance checking framework that entirely relied on NLP for the formalisation of building regulations and their alignment with building design information [26,27]. They developed rules that used syntactic and semantic text features to extract information elements from the International Building Code (IBC) and converted the extracted elements into logic rules. They showed that there is a potential in using NLP to automate the interpretation of building regulations, but they tested their approach only with quantitative requirements of one IBC chapter, and rule-based NLP is usually limited in its scalability since the rules are specialised for a particular text type [28].

The additional use of domain knowledge [29,30] and deep learning [30–33] were explored to improve the information extraction and the information alignment to IFC. Wu et al. [34] enhanced SNACC by improving the generation of logic facts from BIM data through invariant signatures and integrating a user interface for experts to review and fix logic rules generated from regulations.

A common challenge for ACC is the missing information in BIM. While ACABIM and AEC3 request missing information from the user during the compliance checking, CORENET-X defined explicit information requirements necessary to check a BIM. Alternatively, there is an extensive body of research investigating methods for semantic enrichment of building information models [35–37]. Through such methods, missing information required for compliance checks could be inferred automatically, for example, from geometric or visual information.

Another important and complex part of ACC is the execution of the checking rules. While simple quantitative rules could be fully automatically checked by ACC systems such as SNACC, this is already more complex for spatial checks. For example, Li et al. [38] integrated spatial checks in a reasoner to evaluate rules relying on determining the line of sight. Even more problematic are subjective rules that leave room for interpretation. Nawari [39] proposed to treat such cases through fuzzy logic [39] and in ACABIM and AEC3 user inputs are requested. Engineering requirements that require extensive calculations, models, and simulations bring the additional need of integrating external tools, such as done by ACABIM, where the computerised building regulations in LRML are implemented through Compliance Audit Procedure (CAP)s, which can include external tools for evaluation.

### 2.2. Building regulations in LegalRuleML

LRML [40] was developed to bridge the gap between legal text and semantic modelling. Its development facilitates the application of semantic web technologies, including information retrieval and extraction, to the legal domain. Another goal was to support legal reasoning

[1] https://www.solibri.com/
[2] https://up.codes/
[3] https://www1.bca.gov.sg/regulatory-info/building-control/corenet-x
[4] https://www1.bca.gov.sg/regulatory-info/building-control/corenet-x/resources/ifc-sg-resource-toolkit

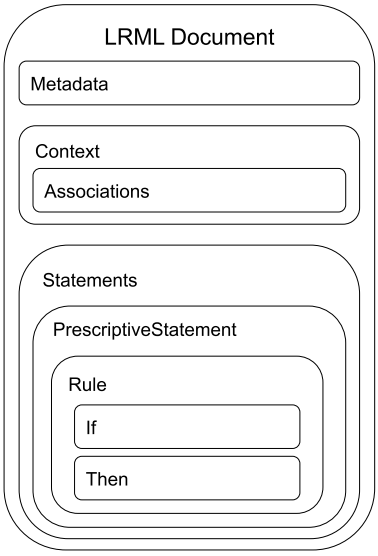



**Fig. 1.** LegalRuleML Document Structure.
*Source:* Adapted from Lam et al. [41].

```
<lrml:PrescriptiveStatement key="
    NZ_NZBC-G13AS1#2.8_r3.4.2.0.1">
  <ruleml:Rule key="NZ_NZBC-G13AS1#2.8
      _r3.4.2.0.1">
    <ruleml:if>
      <ruleml:Expr>
        <ruleml:Fun iri="fuvo:exist"/>
        <ruleml:Atom>
          <ruleml:Var iri="buvo:
              floorWaste"/>
        </ruleml:Atom>
      </ruleml:Expr>
    </ruleml:if>
    <ruleml:then>
      <lrml:Obligation>
        [...]
      </lrml:Obligation>
    </ruleml:then>
  </ruleml:Rule>
</lrml:PrescriptiveStatement>
```

**Fig. 2.** High-level rule definition of the LRML representation for NZBC G13/AS1 Section 3.4.2 [44].

```
Original:
3.4.1 Kerb ramps (see Figure 10) shall
    have:
a) A slope of no greater than 1 in 8,
    and
b) Colour and texture contrast with the
     adjacent footpath.

Requirement 1:
3.4.1 Kerb ramps (see Figure 10) shall
    have: A slope of no greater
       than 1 in 8

Requirement 2:
3.4.1 Kerb ramps (see Figure 10) shall
    have: Colour and texture
       contrast with the adjacent
           footpath.
```

**Fig. 3.** Splitting the enumeration in NZBC D1/AS1 Section 3.4.1 [49] into two requirements.

over semantic knowledge by ensuring that legal domain requirements form the basis of its framework. Accordingly, LRML was developed to represent the semantics and context of legal norms, guidelines and policies. LRML is an XML-based format extending RuleML with operators necessary to represent regulation-specific characteristics, such as deontic operators, defeasibility, and temporality. LRML has independence from logic formalisms and ontology frameworks to allow its use in various domains. Based on the use case, vocabularies or ontologies can be defined independently and referenced from the LRML representation.

Fig. 1 provides the high-level overview of LRML documents, which consist of three main components: metadata, context, and statements. The metadata stores general information about the legal document, the context links LRML rules to specific clauses (e.g., in LegalDocML (LDML)) to ensure isomorphism (i.e., one-to-one mapping), and the statements contain the actual legal clauses, formalised as LRML rules. The statements are the main focus of this paper and are illustrated below using the LRML syntax.

The high-level rule definition of a formalised legal requirement in the LRML format, as produced by Dimyadi et al. [42], is presented in Fig. 2. The logic and semantics are mainly represented using RuleML tags [43]. An exemplary domain-specific encoding is detailed in Section 2.3. Overall, the logic of requirements is captured using an if-then structure.

Statements can be classified as *'PrescriptiveStatement'* (e.g., requirements, exceptions) or as *'ConstitutiveStatement'*, used to specify factual statements, such as definitions. Furthermore, deontic operators such as *'Obligation'*, *'Permission'*, and *'Prohibition'* indicate that a specific conclusion must, may, or must not be met. These classifications are made explicit because of their legal importance.

LRML has been utilised in many domains to formalise regulations. Robaldo et al. [45] formalised the European General Data Protection Regulation (GDPR), Wyner et al. [46] formalised Scottish smoking legislation and regulation, and Governatori et al. [47] the Telecommunication Consumer Protections Code for Business Process Compliance Checking.

Sixteen Acceptable Solution (AS) for the New Zealand Building Code (NZBC), covering the categories 'Stability', 'Protection from Fire', 'Access', 'Moisture', and 'Services and Facilities', were manually translated into LRML by Dimyadi et al. [42].

Six domain experts (i.e., AEC and computer science) were involved in extracting the relevant information and reviewing the extracted information. The translation process started with splitting paragraphs into multiple requirements where necessary. In many cases, this could be done by splitting paragraphs into sentences or by resolving enumerations as exemplified in Fig. 3. Each item in the list was converted into an individual requirement, with the preceding phrase or sentence attached to maintain context. The logical connection between those requirements needs to be captured within the rule-set level of LRML. A similar process called text splitting and stitching was proposed by Zhou and El-Gohary [48].

Alternatively, the experts could decide to merge lists and enumerations into a single LRML rule, where possible, to reduce the necessity of repeating LRML statements. For example, the section in Fig. 3 could be formalised as one LRML rule with two obligations. This adds the benefit for this paper that the used NLP approaches have to deal with lists and enumerations and the resulting increased length of the legal clauses, which was discussed further in Section 5.1.

```
Input:
3.4.2 The floor waste shall have a
    minimum diameter of 40 mm.
Output:
if(expr(fun(exist),
    atom(var(floorWaste)))),
then(obligation(expr(
    fun(greaterThanEqual),
    atom(rel(diameter), var(floorWaste)
        ),
    data(baseunit(prefix(milli), kind(
        metre)), value(40.0)))))
```

**Fig. 4.** LRML rule example - NZBC G13/AS1 Section 3.4.2 [44].

Then, they extracted expressions (i.e., concepts and relationships between these concepts) and identified the logical connectors between those expressions into a tabular format. The Legal Knowledge Model (LKM) dictionary was created to capture the atomic units of the building regulations. A separate dictionary has the advantage that there is no reliance on IFC and other classification systems, and concepts not included in those classifications can be captured in the LRML rule. However, some building-related terms were aligned to the terminology used in classification systems. For example, 'room' was encoded as 'space' as in 'IFCSpace'. For other terms, the classifications served as guidelines on the granularity. For example, with respect to IFC, "accessible ramp" would be represented as a 'ramp' with property 'isAccessible' while "guard rail" would be represented as 'guardRail'.

The tables were then automatically converted into LRML using a purpose-built LRML converter. In addition to generating the LRML rules, the relevant metadata (e.g., jurisdiction, legislation source, legislation version, contexts, associations, and XML versions) was gathered and generated in the LRML syntax.

A great benefit of LRML is its independence from the reasoning engine. Simple rules can be directly evaluated against the building design, complex rules might need additional (potentially reusable) routines, and this solution can be integrated with simulations and other software tools as in ACABIM.

### 2.3. Transformer-based semantic parsing into LegalRuleML

Semantic parsing is the process of transforming natural language into a formal representation of its meaning. Such representations range from semantic representations such as Abstract Meaning Representation (AMR) to logical representations such as First-Order Logic or Lambda Calculus, query languages such as Structured Query Language (SQL) and SPARQL Protocol And RDF Query Language (SPARQL), and programming languages such as Python and Java.

Fuchs et al. [6] proposed to use a Text-to-Text Transfer Transformer (T5) model [50] pre-trained on AMR parsing to translate building regulations into the short version of LRML shown in Listing 4; the XML-based representation was compressed using parentheses to make the representation less verbose and better suited for generative neural networks such as T5.

T5 employs the transformer architecture proposed by Vaswani et al. [51]. The model encompasses an encoder to generate a latent representation of the input and a decoder to combine the output generated so far with the input representation through an attention mechanism and generate the following output token. The primary mechanism to generate the latent representation is self-attention, where each token is contextualised by incorporating the relevant information of all other tokens in the sequence. The scaled dot-product attention is calculated as shown in Eq. (1), where $d_k$ is the dimension of the keys. Three trainable weight matrices for queries $Q$, keys $K$, and values $V$ are utilised to calculate the weighted sum of the input information.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

The speciality of the T5 model is that it formalises each task into a text-to-text format by formulating the task with an initial prompt, e.g., "translate English to German", and all answers are transformed into a textual representation, e.g., "True" and "False", or "cat" and "dog", rather than binary values 0 and 1 for classification tasks. This strategy makes the model easily applicable to a broad range of tasks and suitable for multi-task learning, i.e., learning to do multiple tasks in parallel.

### 2.4. Intermediate representations

Significant improvements in semantic parsing of natural language expressions into SQL and SPARQL were achieved by coarse-to-fine-grained parsing [52] and using reversible and lossy IRs [53]. Both Dong and Lapata [52] and Herzig et al. [53] followed the strategy to generate a simplified representation as an intermediate step towards the final output representation. They retained the benefits of seq2seq approaches, such as reducing cascading errors and information loss and utilising the strength of pre-trained language models while reducing the complexity of the task.

Any representation on the spectrum between the input text and the target representation is considered an IR. Representations that have the potential to be generated more easily or that can support the generation of the target representation are of interest. IRs can be related to hierarchical information processing, which was applied to building regulations by Zhou and El-Gohary [48] and Zhang and El-Gohary [54]. Considering the hierarchically complex structure of legalese, IRs are a promising candidate to support the semantic parsing of building regulations.

Herzig et al. [53] distinguished between reversible and lossy IRs. Reversible IRs are deterministic transformations of the original representation that can be undone by applying the reverse function. An example of such an IR for the LRML representation is combining entities and properties using a dot notation: *'atom(rel(width), var(wall))'* → *'atom(wall.width)'*.

Lossy IRs are further distant from the target representation. For example, given a target representation, the lossy IR can be generated by removing certain information. To generate Python code, Dong and Lapata [52] replaced variable names with the *'NAME'* placeholder and values with their data type (e.g., *'NUMBER'* or *'STRING'*). Alternatively, a lossy IR can be generated from the input text. For example, entities could be extracted from the source text to be transformed into a formal representation. An alignment or a reliable matching algorithm between the input text and the target representation might be required to identify which entities are needed to create the formal representation. Otherwise, the work-intensive task of manually creating a ground truth must be carried out.

The closest work to this paper was by Ferraro et al. [55], who used the coarse-to-fine-grained parsing by Dong and Lapata [52] to semantically parse RegTech, a dataset with a lambda calculus representation of 140 sentences from general regulations, including building regulations, shown in Fig. 5. Compared to their work, this paper considers representations that are further distant from the input text and have more fine-grained levels of abstraction rather than parsing the clause into a representation that is only used as a step towards producing the machine-interpretable rules. Furthermore, this paper utilises pre-trained transformer models rather than training a Long Short-Term Memory (LSTM) model from scratch and explores a broad range of settings to investigate the potential of IRs.

```
Input:
A large building is any building with a
    net lettable area greater than 300
    m2.
Output:
lambda $0 (if (a large building:$0)
   then (is any building with a
   lettable area greater than ($0 300
   m2)))
```

**Fig. 5.** RegTech example [55].

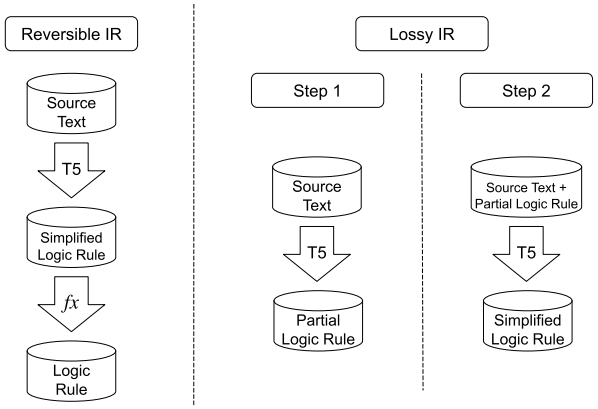



**Fig. 6.** Using IRs for semantic parsing.

## 3. Methodology

Similar to Herzig et al. [53], the distinction between reversible and lossy IRs is drawn for the proposed training method as visualised in Fig. 6. While reversible IRs can be generated with a single Machine Learning (ML) model and the original formal representation can be restored through applying the reverse function, the lossy IR strategy requires one to train two separate ML models. One for generating the IR and a second one to generate the final formal representation or alternatively the reversible IR.

Reversible IRs strongly depend on the representation language. Logic languages such as Prolog have less need for optimisation than verbose XML-based formats such as RuleML and LegalRuleML. Specific simplifications for reversible IRs were investigated in the context of the LRML representation in Section 3.1. In general, these simplifications need to be fully reversible so that the original representation can be deduced. Based on the evaluation metrics used, either the original or the simplified representation could be evaluated. Since the reversible representation had the benefit of being shorter and less complex, it was advantageous to establish this representation first and reuse it as the basis for developing lossy IRs.

A range of factors make semantic parsing of building regulations a difficult endeavour. Regulatory requirements can be very complex, with deeply nested logical connections, insertions of adverbial clauses, multiple subordinate clauses, double negations and other syntactic characteristics uncommon in general language [56]. Such difficulties could be remediated by allowing the semantic parser to process the text in multiple steps, giving it effectively more space and time for computations, for example, by simplifying the text or extracting only parts of the information. In particular, where the terminology of the formal representation is dissimilar from the textual form, it can be helpful to treat the entities and relations separately. Nevertheless, if Step 1 in Fig. 6 involves removing information from the original input, the problem of cascading errors re-arises. To limit this problem, the extracted information or partial logic rule can be appended to the source text and act as an additional input to the semantic parser (rather than replacing the source text) while generating the final formal representation.

```
for epoch = 1 to n do
    for each batch in training_set do
        Forward pass through the model
        Calculate loss
        Backwards pass
        Update learning rate scheduler
        Zero gradients
    end for
    for each batch in validation_set do
        Generate outputs
    end for
    Calculate validation metrics
end for
Load the best model
for each batch in test_set do
    Generate outputs
end for
Calculate evaluation metrics
Log results
```

**Fig. 7.** Training and evaluating a transformer model.

Fig. 7 shows the general training procedure of each neural network. The model is trained for $n$ epochs. In each epoch, the training set is split into random batches. Each batch is passed through the model (i.e., forward pass) to produce the output probabilities. These probabilities are compared to the ground truth data, the cross entropy loss is calculated, and the model weights are updated in a backward pass with respect to that loss. After each epoch, the model is validated using the validation set. The generation parameters, including beam search, are used as they are during inference. The evaluation metrics are calculated (i.e., Bi-Lingual Evaluation Understudy (BLEU) Papineni et al. [57] and F1 score, as defined in Fuchs et al. [6]). When the maximum number of epochs $n$ is reached, the training is stopped. The best model, with respect to the validation set, is loaded and evaluated on the test set, for which the evaluation metrics are reported in Section 4.

Section 3.1 proposes a selection of reversible IRs for the verbose LRML representation shown in Listing 4. The resulting reversible IR is used throughout the rest of the paper. Section 3.2 introduces suitable lossy IRs for the LRML representation. Additionally, creating IRs from the input clause was exemplified by paraphrasing the legal clauses in Section 3.4. Then, the potential of using the lossy IR training strategy to allow a model to fix mistakes in the generated logic rules was explored in Section 3.5.

### 3.1. Reversible intermediate representations

When using reversible IRs, a simplified representation is predicted instead of the original representation. Then, the reverse function is applied to evaluate the quality of the prediction. Accordingly, there is a common measure to compare the translation performance into the original representation and the IR.

Reversible IRs strongly depend on the representation language. A closer investigation was conducted for LRML. Reversible IRs were identified by inspecting the LRML representation used for semantic parsing as shown in Fig. 4 and the processes used to create the LRML documents initially [42] as described in Section 2.2. These IRs are also related to the linearisation efforts employed to form the LRML

```
Input:
3.4.2 The floor waste shall have a
    minimum diameter of 40 mm.
Output:
if( exist( floor waste)),
then( obligation( greater than equal(
    floor waste. diameter,
    40 mm)))
```

**Fig. 8.** LRML rule example from NZBC G13/AS1 Section 3.4.2 [44] with all reversible simplifications applied and spaces added for improved tokenisation following Fuchs et al. [7]. See Fig. 4 for the original representation.

dataset in Fuchs et al. [6]. Following is a list of the proposed LRML transformations, which were applied and evaluated systematically in Section 4.1.

- **'unit':** Dimyadi et al. [42] used a LRML converter for the conversion of data extraction spreadsheets to LRML, which included an automatic unit conversion to enrich the LRML rules with additional metadata. By reverting the unit conversion, this overhead was removed from the semantic parsing task, and the existing unit conversion was applied after generating the intermediate representation: *'data(baseunit(prefix(milli), kind(metre)), value(40.0))' → 'data(40 mm)'*
- **'and'/'or'**: Many occasions were identified where expressions needed to be repeated for different data values, for example, when translating *"electric and gas storage water heaters"*, where two types of *storage water heaters* are specified. To avoid this redundancy, conjunctions and disjunctions were allowed in the data field, and consecutive expressions with such duplication were combined. For example: *'and(expr(fun(is), atom(rel(type), var(storageWaterHeater)), data(electric)), expr(fun(is), atom(rel(type), var(storageWaterHeater)), data(gas)))' → 'expr(fun(is), atom(rel(type), var(storageWaterHeater)), data(and(electric, gas)))'*
- **'atom'**: Using atoms as an additional container inside expressions to bundle subjects ('var') and their properties ('rel') increased the length further. This was addressed by expressing subjects-property pairs using a dot notation: *'atom(rel(diameter), var(floorWaste))' → 'atom(floorWaste.diameter)'*
- **'expression'**: The remaining expressions were simplified further by utilising the function and relation terms *'fun'* as predicate and inserting the atom and data values as arguments: *'expr(fun(greaterThanEqual), atom(floorWaste .diameter), data(40 mm)' → 'greaterThanEqual(floorWaste.diameter, 40 mm)'*
- **'loop':** Fuchs et al. [7] identified loops as especially problematic due to their complex representation. Accordingly, the following simplification was applied: *'expr(rulestatement(forEach(...)), appliedstatement(is(...)))' → 'loop(forEach(...), is(...))'*

These simplifications resulted in the shorter and more compact representation shown in Fig. 8 with the following expected benefits for semantic parsing: (1) faster training and inference, (2) fewer syntactic mistakes, (3) reduced complexity, and (4) increased training stability. Section 4.1 investigates those benefits by evaluating the LRML dataset on the reversible IRs.

### 3.2. Lossy intermediate representations

Given the simplified LRML representation, the lossy IRs were generated by dropping or extracting some parts of the representation. Separate models were trained as visualised in Fig. 6 and demonstrated in Fig. 9 to predict the lossy IR given a legal clause (Model 1) and the simplified LRML given the predicted lossy IR and the legal clause (Model 2). As an alternative, Model 2 was trained using the oracle IR (i.e., ground truth IR) rather than the predicted IR as additional input, which might help the model to learn the correct utilisation of the IR rather than having a noisy training signal. However, during inference, Model 2 has to deal with noisy outputs. This motivates the third setup, where Model 2 was taught to predict the final representation from both the predicted IR and the oracle IR, which were concatenated for the training. This experiment indicates whether it is more critical for Model 2 to receive a correct IR during training or better to prepare Model 2 for data closer to what is expected at inference time, which is related to teacher forcing a common technique used for training neural networks [58].

Pre-training a model on a related task can benefit the semantic parsing task, as observed in the multi-task learning experiments in Fuchs et al. [6] for building regulation and in Raffel et al. [50] and Aribandi et al. [59] for general domain tasks. In an additional experimental setup, instead of refining two separate T5 models for the tasks, Model 1 was reused as a starting point for training Model 2 and refined to predict the final formal representation, possibly utilising patterns learned for the earlier task of generating the IR.

Finally, an investigation was conducted on whether and how well Model 2 utilises the IR. The evaluation included three versions of the test set with different inputs to address this question:

1. Legal clause plus predicted IR
2. Legal clause plus oracle IR
3. Legal clause only.

As an intuition, the parsing performance should increase in the first case if the predicted IR is good enough, the quality should be close to perfect given the oracle IRs (depending on the closeness of the IR), and in the third case, the model should still be able to predict meaningful formal representations, given it does not entirely rely on the IR.

Lossy IRs are less dependent on the representation language. So, the lossy IRs below explored for the LRML representation might be similarly usable for other formats.

#### 3.2.1. Entity extraction

Most previous approaches to automate the formalisation of building regulations were based on entity extraction. A natural way to begin the manual formalisation is to identify the main entities and properties in a legal clause and their logical connections. A unique set of all subjects, properties, and objects were extracted from the LRML representation and used as IR. The entities from the LRML representation, rather than from the legal clause, were chosen due to the missing alignment data between the representations. Fig. 9 shows how the models are trained using this IR.

### 3.3. Expressions only

Identifying the correct logical connections between expressions is particularly important. Having expressions falsely assigned to the antecedent or consequent, having expressions connected with conjunctions rather than disjunctions, or missing negations can result in rules being not or falsely triggered and compliance issues being falsely reported or missed. There is a high likelihood for such cases to happen in ML because such labels are often unbalanced, and a model might learn to predict the more common case. This difficulty was counteracted by predicting only the expressions in the first step, which meant removing *'if', 'then', 'and', 'or', 'not', 'obligation'*, and similar keywords. The hypothesis was that it would be easier to identify those logical connections after all expressions were identified (i.e., *'exist(floorWaste)'* and *'greaterThanEqual(floorWaste.diameter, 40 mm)'* for the recurring example in Fig. 9).

```
Model 1:
Input:
extract LegalRuleML entities: G13AS1 3.4.2 The floor waste shall have a
    minimum have a minimum diameter of 40 mm.
Output:
floorWaste, diameter, 40 mm

Model 2:
Input:
translate English to LegalRuleML: G13AS1 3.4.2 The floor waste shall have a
    minimum diameter of 40 mm.<sep> floorWaste, diameter, 40 mm
Output:
if(exist(floorWaste)),
then(obligation(greaterThanEqual(floorWaste.diameter, 40 mm)))
```

**Fig. 9.** Example input and output for lossy IRs. NZBC G13/AS1 Section 3.4.2 [44].

### 3.4. Paraphrases as intermediate representation

Long, complex, nested legal clauses can increase the difficulty of the semantic parsing task. Instead of generating the IR from the formal representation, the IR could be created from the legal clause. For example, simplifying the natural language legal clause might enhance the parsing quality. However, there is the difficulty of establishing a ground truth automatically. This task could be carried out by Large Language Models (LLMs) such as General Pre-trained Transformer (GPT) [60] (see Fuchs et al. [61] for more information on this topic). However, for the exploration in this paper, paraphrases were constructed manually in a way similar to controlled natural languages by applying the following transformations to the legal text:

- **If-then structure**: Reorder the clauses and embed them into an if-then structure. This step transforms the legal clause into a high-level natural language rule closer to the formal representation. E.g., *"The floor waste shall have.."*. → *"If a floor waste exists, then..."*
- **Remove unnecessary text**: Remove text not represented in LRML, including statements describing the reason for a particular legal requirement and the simplification of long phrases. This step removes the filtering step from the semantic parsing task, allowing the semantic parser to formalise all parts of the resulting paraphrase.
- **Coreference resolution**: Repeat entities if references occur over a longer distance or in the premise and conclusion. This can support the correct identification of the subject of an expression. E.g., *"The floor waste shall have..."* → *"If a floor waste exists, then the floor waste shall..."*
- **Scope of conjunctions and disjunctions**: How disjunctions and conjunctions are nested in long sentences can become unclear. This problem was remediated by repeating the logical connectors and indicating the start of disjunctions using phrases such as *"either...or...or..."*.
- **References**: References to documents, clauses, tables, and figures are prevalent in legal text. Representation languages commonly fully specify the document code for such references to guarantee a unique identifier, while in legal clauses, references to the same document do not require this information. Document codes were added to such instances to support the semantic parser. E.g., *"this document"* → *"G13/AS1"*, *"Table 4.5"* → *"C/AS2 Table 4.5"*.

The predicted paraphrases are expected to retain all information necessary to produce the formal representation. Accordingly, in contrast to the lossy IRs in Section 3.2, the paraphrases were not appended to the Model 2 input, but Model 2 was trained to predict the formal representation directly from the paraphrased clause, as in Herzig et al. [53]. While this decision puts more importance on the quality of the paraphrases, it removes the need to share attention between the two inputs.

### 3.5. Self-reflection

In Section 3.2, expressions were isolated as IR, leaving out logical connections. This IR is very close to the target representation, which raises the questions:

- What if the target representation is appended instead of the IR, and the model has the chance to correct mistakes made during the initial predictions?
- Can the model recognise such mistakes given the additional context from the right-hand side?

These questions relate to the recent successes of GPT-4 [60] in showing signs of self-reflection with the ability to improve its previous outputs by being prompted about the correctness of its responses [62]. While such behaviour might exist for LLMs, the experiments below evaluated whether the smaller T5 models can improve themselves and whether this method can make different generation models "collaborate".

These questions were investigated by training two separate models as described in Section 3.2. Model 1 predicts the complete formal representation as an "IR", given the regulation clause. Model 2 predicts the improved formal representation, given the regulation clause and the formal representation predicted by Model 1. The main intention was to train Model 2 to correct mistakes made by Model 1. However, compared to lossy IRs, in this experiment, training Model 2 with oracle IRs was not expected to be beneficial since such training might promote copying the solution (i.e., the IR). Similarly, the predicted IRs were generated by the fully trained model, which tends to overfit the training data. Accordingly, the predicted IRs for the training clauses closely resembled the original LRML rules, with few errors requiring correction. This scenario promotes copying the IR, as encountered when training with oracle IRs for the expression IR in Section 3.2. A separate experiment was conducted, where IR predictions for the training data were generated before overfitting the training data (i.e., "Without Overfitting" in Table 5). Model 1, after three epochs, was chosen to generate the training IRs. The validation after three epochs was the point in time when the training loss became lower than the validation loss in most cases. As in previous experiments, the test and validation IR predictions were from the best Model 1, which was the same for both scenarios.

## 4. Results

To evaluate these reversible IRs (and the lossy IRs in the following sections), T5-AMR was trained as per Fuchs et al. [6] using the new representation as the target. A custom training loop was implemented

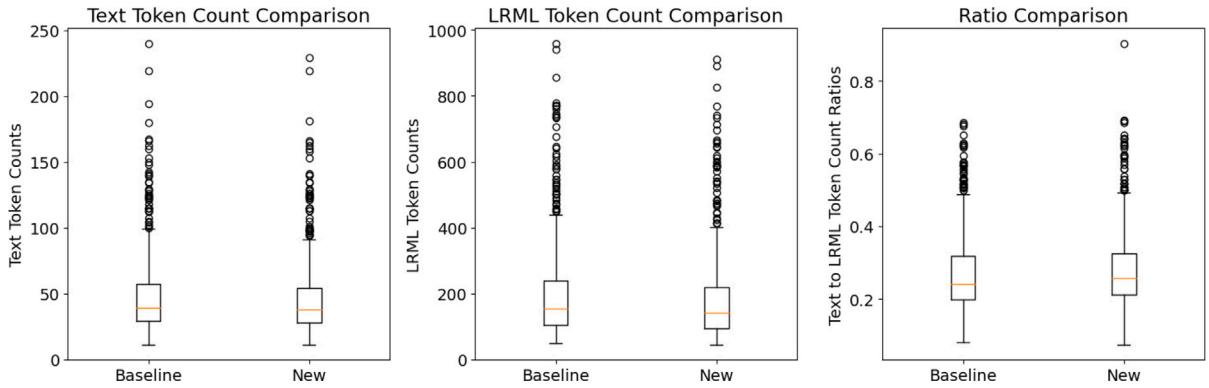



**Fig. 10.** Token counts before and after evaluation-based data cleansing. The tokenisation was conducted using the T5-base tokeniser. Token counts are displayed for the text and LRML. Additionally, the text-to-LRML token count ratio is provided.

following the Huggingface guidance[5] to allow for more flexibility in the training process. The following hyperparameters were changed compared to Fuchs et al. [7] to achieve more stable training: Learning rate: 1e-4, Batch size: 8, Epochs: 20, No Early Stopping.

Following Fuchs et al. [7], BLEU and F1 scores averaged over three runs with different random seeds (i.e., 43, 44, and 45) are reported. The BLEU score was calculated using the Huggingface evaluate library[6] and the default parameters for the BLEU metric (i.e., maximum n-gram order = 4).

Additionally, the generation arguments and procedure were optimised to limit the problem of repeated entities and expressions:

- Repetition penalty = 1.2: Repetition penalty reduces the probability of already generated tokens. Since the LRML representation naturally requires many repetitions, this parameter was increased only slightly from its default value of 1.
- No repeat n-gram size = 9 or 13: This parameter prevents n-grams over the specified threshold from being generated multiple times. Repetitions are strictly prevented, but $n$ must be large enough to allow intended repetitions, such as expressions with the same subjects and relations but different objects. Accordingly, the threshold was set to $n = 13$ before applying the expression transformation detailed in Section 3.1 and $n = 9$ after this transformation.
- Beam size = 3: Beam size was set to the best value according to Fuchs et al. [6].
- Post-processing: Following common practice in semantic parsing, post-processing was introduced to fix the tree structure based on the *'if'* and *'then'*-keywords. If the *'then'*-node appears as a child node of the *'if'*-node, the corresponding sub-tree was removed and re-attached as the sibling of the *'if'*-node, following the assumption that the semantic parser did not close all required brackets for the *'if'*-branch

The analysis in Fuchs et al. [7] provided deep insights into the dataset, showed the importance of semantically consistent translations, investigated the problematic aspects of the LRML representation for its use for semantic parsing, indicated that these problems might not be automatically resolvable, and showed that especially rules from the NZBC B1/AS1 [63] were problematic.

These issues were resolved with an additional data cleansing round, which made use of the LRML editor, which was introduced in Fuchs et al. [64]. This editor implements and supports some of the enhancements proposed in Fuchs et al. [7], which eased the manual review and cleansing process. The data cleansing included the correction of the following aspects:

- Translation consistency: The same phrases are translated the same way. The most common translations were applied throughout the dataset.
- Granularity: The granularity of concepts was aligned to IFC, UniClass and Omniclass. If a multi-word expression could not be mapped to any of those classifications, it was split into more atomic concepts that could be mapped as described in Section 2.2.
- Logical consistency: It was ensured that if-statements contain only selection and applicability criteria [65], and missing relations between elements were added.
- LRML order: The order of LRML expressions within disjunctions or conjunctions was aligned to the legal text.
- Remove clauses deemed untranslatable: Clauses such as the ones in B1/AS1 describing textual changes to referenced standards were removed. An example of such a clause is: *"11.1 AS/NZS 3725 subject to the following modifications: Clause 10.3 After the words 'the test load' add 'or proof load'."* [63]. Rather than indicating textual changes, the LRML rule for the referenced clause should be amended.
- Alignment fixes: Title information was only provided where used in the LRML rule. This change should make the implicit knowledge added in Fuchs et al. [7] more effective.
- Complex expressions: Using variables and loops was restricted to unavoidable cases. An example where variables were necessary is the comparison of dimensions relative to other building elements.
- Prohibitions: In deontic reasoning, there is an equivalence between a Prohibition $P(x)$ and an Obligation $O(\neg x)$ where $\neg x$ is the negated predicate of $P$. Utilising this equivalence, all Prohibitions $P$ were modelled consistently as $O(\neg x)$.

Applying these changes resulted in the final LRML dataset, which contains 718 LRML rules. The slight reduction was the result of removing and combining some rules. Fig. 10 shows the resulting token counts, which were slightly decreased for both the clauses and the LRML rules. However, the text to LRML token count ratio slightly increased. The outlier in this plot represented a case where the model had to ignore the objective of the clause and referenced examples and only formalise the actual requirements.

The LRML dataset was split into training, validation and test splits. Two different test splits were considered: A Random Test Split (RTS) and a Document Test Split (DTS). For the RTS, the dataset was split 8:1:1 for training, validation, and testing (i.e., 576/71/71 samples). For the DTS, the LRML rules for the same AS as used in Fuchs et al. [7] were chosen. However, since there were fewer samples for these documents, training and validation samples were randomly selected to match the RTS training and validation set sizes (i.e., 576/71/55 samples).

For completeness, Table 1 shows the parsing performance on the final dataset compared to the LRML dataset as used in Fuchs et al. [7] but with the training–validation–test splits established in this paper.

[5] https://github.com/huggingface/transformers/blob/main/examples/pytorch/translation/run_translation_no_trainer.py

[6] https://huggingface.co/spaces/evaluate-metric/bleu

**Table 1**
Establishing the Baseline. The average Runtime in seconds and the BLEU and F1 scores for the DTS and RTS with standard deviations in brackets are reported. The new baseline (i.e., the last row) was established on the cleansed dataset, using the additional generation arguments to avoid repetition and post-processing to fix missing parenthesis. The old baseline uses the dataset as in Fuchs et al. [7].

| IR | Runtime | DTS | | RTS | |
|---|---|---|---|---|---|
| | | BLEU | F1 score | BLEU | F1 score |
| Old Baseline | 7619 | 54.6% (6.5) | 45.2% (1.7) | 68.8% (2.9) | 60.0% (2.1) |
| + Cleansed Dataset | 3060 | 60.1% (2.4) | 54.9% (2.6) | 69.5% (3.9) | 63.3% (1.3) |
| + Generation Arguments | 4672 | 65.1% (1.1) | 55.9% (0.3) | 68.4% (0.6) | 64.4% (0.4) |

**Table 2**
Results for the reversible IRs. The average Runtime in seconds and the BLEU and F1 scores for the DTS and RTS with standard deviations in brackets are reported. The baseline included the use of generation arguments to avoid repetition, post-processing, and the cleansed dataset.

| IR | Runtime | DTS | | RTS | |
|---|---|---|---|---|---|
| | | BLEU | F1 score | BLEU | F1 score |
| Baseline | 4672 | 65.1% (1.1) | 55.9% (0.3) | 68.4% (0.6) | 64.4% (0.4) |
| + Unit | 4063 | 66.3% (1.1) | 57.0% (2.0) | 71.2% (1.0) | 65.2% (0.6) |
| + And/Or | 2437 | 62.9% (2.0) | 55.6% (1.1) | 71.6% (1.9) | 65.3% (0.9) |
| + Atom | 4703 | 64.9% (3.5) | 55.6% (1.2) | 71.7% (1.6) | 65.2% (0.9) |
| + Atom (w/o And/Or) | 4256 | 68.1% (1.0) | 58.5% (0.8) | 70.5% (1.4) | 65.4% (1.0) |
| + Expression | 1484 | 62.7% (5.0) | 56.4% (1.2) | 71.7% (0.7) | 65.8% (1.3) |
| + Expression (w/o And/Or) | 1952 | 66.2% (0.2) | 55.5% (0.6) | 71.6% (3.0) | 64.9% (1.9) |
| + Loop | 1270 | 64.7% (0.3) | 55.4% (1.2) | 71.1% (1.1) | 66.4% (0.3) |
| + Loop (w/o And/Or) | 1548 | 66.3% (2.1) | 56.6% (0.8) | 71.2% (0.4) | 65.9% (0.6) |

**Table 3**
Results for the lossy IRs in F1 scores. Two IR experiments are reported: Entities and Expressions (EXPR).

| IR | Model | Model input | Oracle IR | | w/o IR | | Predicted IR | |
|---|---|---|---|---|---|---|---|---|
| | | | DTS | RTS | DTS | RTS | DTS | RTS |
| E | Model 1 | Original | – | – | – | – | 45.3% (BLEU) | 62.9% (BLEU) |
| N | Model 2 | Prediction | 75.6% | 80.4% | 38.6% | 36.4% | 55.5% | 68.2% |
| T | (New | Oracle | 77.0% | 81.8% | 35.7% | 34.5% | 55.1% | 68.7% |
| I | Model) | Combined | 77.0% | 82.2% | 40.4% | 38.5% | 55.0% | 69.7% |
| T | Model 2 | Prediction | 73.9% | 79.2% | 51.9% | 58.1% | 55.6% | 69.6% |
| Y | (Refine | Oracle | 76.0% | 82.3% | 48.2% | 55.6% | 55.0% | 68.9% |
| | Model 1) | Combined | 74.9% | 82.0% | 51.5% | 59.4% | 55.1% | 71.0% |
| | Model 1 | Original | – | – | – | – | 43.3% | 48.9% |
| E | Model 2 | Prediction | 92.0% | 94.9% | 0.0% | 0.0% | 56.4% | 65.0% |
| X | (New | Oracle | 97.6% | 97.5% | 0.0% | 1.0% | 56.0% | 64.4% |
| P | Model) | Combined | 97.0% | 96.9% | 0.0% | 0.3% | 56.2% | 65.1% |
| R | Model 2 | Prediction | 72.5% | 85.3% | 53.2% | 62.1% | 57.1% | 66.9% |
| | (Refine | Oracle | 97.5% | 97.6% | 26.6% | 32.1% | 56.0% | 64.5% |
| | Model 1) | Combined | 91.1% | 94.8% | 51.8% | 62.9% | 56.3% | 66.4% |

It is noticeable that the semantic parsing performance was especially improved for the document split of the cleansed dataset, decreasing the gap between both test sets. Furthermore, the training runtime was decreased significantly, potentially due to the LRML rules generated during validation being shorter and closer to the ground truth. The updated generation arguments and post-processing improved the baseline by about 1% F1 score and significantly stabilised the results but increased the training runtime by a factor of 0.5. However, there might be a trade-off between intended and unintended repetition, and it should be re-evaluated if the n-gram repetition constraint is still beneficial with better and larger models.

### 4.1. Reversible intermediate representations for LegalRuleML

The reversible IR transformations were applied step by step. The experiments were conducted with and without the *'and'/'or'* transformation since this transformation was the most questionable. Table 2 shows the results of the experiments. The most significant improvement through the reversible IRs was the training time, which could be decreased to nearly a quarter of the new baseline's runtime. Furthermore, the F1 score for the RTS increased by 2% when applying all transformations. The *'unit'* and *'expression'* transformations achieved the most significant improvements. Evaluating against the DTS yielded more counter-intuitive and unstable results. The *'atom'* transformation with the *'and'/'or'* transformation led to worse outcomes, but without the *'and'/'or'* transformation led to the best result. The inconsistent results for the DTS might stem from the smaller test set size or the high sample similarity within the test set. Due to the issues with the DTS and due to F1 scores being used for the best model selection, the RTS F1 score was treated as the primary metric, and the LRML representation with all transformations (i.e., "+ Loop") was selected for the experiments in the remaining paper.

### 4.2. Lossy intermediate representations for LegalRuleML

The effectiveness of the two lossy IRs (i.e., entities and expressions) was evaluated with the LRML dataset. The training of the T5-AMR

**Table 4**
Results for the paraphrases (PARA) as IR measured in F1 scores (if not specified differently).

| IR | Model | Model input | Oracle IR | | Predicted IR | |
|---|---|---|---|---|---|---|
| | | | DTS | RTS | DTS | RTS |
| | Model 1 | Original | – | – | 35.6% (BLEU) | 56.0% (BLEU) |
| P | Model 2 | Prediction | 71.4% | 82.1% | 53.8% | 66.7% |
| A | (New | Oracle | 71.6% | 81.5% | 52.7% | 66.4% |
| R | Model) | Combined | 73.0% | 84.8% | 53.9% | 68.9% |
| A | Model 2 | Prediction | 69.2% | 81.1% | 53.3% | 67.7% |
| | (Refine | Oracle | 70.6% | 80.6% | 53.0% | 66.7% |
| | Model 1) | Combined | 71.7% | 83.8% | 54.6% | 68.9% |
| – | T5-AMR | Original & Oracle | – | – | 56.5% | 70.1% |

**Table 5**
Results for the self-reflection experiments measured in F1 score. Self-reflect: LRML IR generated with the best IR model; Without overfitting: LRML IR generated after three epochs.

| IR | Model | Model input | Oracle IR | | w/o IR | | Predicted IR | |
|---|---|---|---|---|---|---|---|---|
| | | | DTS | RTS | DTS | RTS | DTS | RTS |
| – | Model 1 | Original | – | – | – | – | 55.4% | 66.4% |
| Self-reflect | Model 2 | Prediction | 98.6% | 97.4% | 0.0% | 0.0% | 55.3% | 66.3% |
| | (New | Oracle | 99.3% | 98.5% | 0.0% | 0.0% | 55.5% | 66.4% |
| | Model) | Combined | 98.9% | 98.2% | 0.0% | 0.0% | 55.4% | 66.5% |
| | Model 2 | Prediction | 76.1% | 85.7% | 53.1% | 66.7% | 55.9% | 67.3% |
| | (Refine | Oracle | 98.8% | 98.3% | 45.2% | 54.2% | 55.4% | 66.4% |
| | Model 1) | Combined | 90.8% | 95.0% | 52.3% | 65.5% | 55.5% | 67.7% |
| Without over-fitting | Model 2 | Prediction | 73.6% | 87.5% | 0.0% | 0.0% | 56.4% | 65.5% |
| | (New | Oracle | 99.3% | 98.5% | 0.0% | 0.0% | 55.5% | 66.4% |
| | Model) | Combined | 98.9% | 98.2% | 0.0% | 0.0% | 55.4% | 66.5% |
| | Model 2 | Prediction | 59.8% | 73.3% | 56.4% | 71.0% | 56.5% | 70.9% |
| | (Refine | Oracle | 98.8% | 98.3% | 45.2% | 54.2% | 55.4% | 66.4% |
| | Model 1) | Combined | 90.8% | 95.0% | 52.3% | 65.5% | 55.5% | 67.7% |

models was conducted as for the reversible IRs in Section 4.1. Per IR, either two separate models were trained, or the same model was trained sequentially (also termed multi-task). Additionally, Model 2 was trained using predicted IRs, oracle IRs, or predicted and oracle IRs combined. Testing Model 2 using predicted IRs, oracle IRs, and without IRs gave an additional indication of what the model learned during training.

Table 3 reports the F1 scores for the DTS and RTS. BLEU scores and standard deviations have been omitted to keep the results comprehensible. Using expressions as IR improved the DTS performance by 1.7%, but the RTS performance only by 0.5% in the best setup, potentially due to the F1 scores for the IR being closer to the LRML F1 score in the case of the DTS. In contrast, for RTS, there were consistently large improvements using entities as IR, with up to 4.6% for the multi-task Model 2 trained on both the predicted and the oracle entities. In most cases, continuing the training from Model 1 worked better than training a new model from scratch. This might be the case since Model 1 has already learned to use the regulation clause for predicting the IR and does not rely too much on the appended IR after refinement. This is especially noticeable for the expression IR since the separately trained model had 0% F1 scores when removing the IR. Also, this model scored close to 100% given the oracle expressions, indicating it primarily relied on copying the inputs, but it made some mistakes inserting the logical connectors.

A significant challenge is the correct generation of the IR. For example, Model 1 for entity extraction was evaluated using BLEU scores. Extracting the entities for the DTS had only a BLEU score of 45.3%, while for the RTS, the BLEU score was 62.9%. These BLEU scores indicate that the RTS LRML rules might be closer to the legal text than the DTS LRML rules. This might also explain why this IR was only helpful for the RTS. The low BLEU scores and the oracle results indicate that enhancing entity extraction might improve the parsing performance. Integrating the dictionary, which was established during the creation of the initial LRML dataset, into the entity extraction is a promising direction for future work.

### 4.3. Paraphrases for legal clauses

Table 4 reports the results for using the prediction of paraphrases as an intermediate step towards producing LRML. The same combinations of models and model inputs were used as in the evaluation in Section 3.2. The test set without IR as input was omitted because this case was not directly applicable for paraphrase-only inputs, and the intended use of this ablation to identify the utilisation of the appended IR was non-existent.

Using paraphrases as IR gave improvements of up to 2.3% for the RTS. It is unclear if the main reason for the improvement was the IR or the additional training data since having two versions of input resulting in the same output could also benefit learning. Training a model with both the original and paraphrased clauses shows evidence that this might be the case (i.e., 70.1% F1 score for the RTS).

The DTS performance deteriorated, possibly due to the weak paraphrasing performance: 35.6% and 56.0% BLEU for DTS and RTS, respectively. Nevertheless, F1 scores of up to 84.8% with the oracle paraphrases indicate that a simplified systematic formulation of legal texts, or potentially a natural language rewriting, might allow legal experts to produce LRML or other formal representations without much knowledge engineering experience. This could also be integrated into a user-friendly semi-automated LRML translation interface.

### 4.4. Self-reflection for LegalRuleML

Table 5 reports on the experimental results and gives an indication of whether the T5 model was able to correct mistakes. Similar to the previous results, a newly trained Model 2 does not seem to learn anything but how to copy the LRML rules. Also, the multi-task model trained with the oracle LRML rules mostly learned to copy the inputs, indicated by having close to perfect F1 scores for the oracle IRs and the same F1 scores as Model 1. But in contrast to separately trained models, the multi-tasking model did not entirely unlearn predicting the LRML when no IR was given. Slight improvements were achieved in the multi-tasking setup and training with the predictions and combined IRs.

The overfitting hypothesis was strengthened through positive results for the second IR (i.e., without overfitting), where the F1 score increased to 70.9% with the IR LRML rules predicted after three epochs. In this experiment, training with the oracle and predictions decreased the performance since the model started to copy the inputs.

However, the best-performing model was better when removing the predicted LRML rules during inference and only slightly better with the oracle LRML. So, whether the model made meaningful corrections or whether the appended LRML rules improved the training process by acting as an exemplary translation is questionable and needs further analysis in future work.

## 5. Discussion

Applying the reversible IR described in Section 3.1 reduced the number of sub-word tokens of the LRML rules from an average of 181 and a maximum of 911 to an average of 82 and a maximum of 408.

```
Input:
translate English to LegalRuleML:
    G12AS1 Piped water supply system;
    Covers shall be provided to: All
    tanks located in roof spaces to
    prevent condensation damaging
    building elements.

Predicted IR:
pipedWaterSupplySystem, tank, location,
     roofSpace, cover
Prediction with entity IR:
if(has(pipedWaterSupplySystem, tank)),
    then(obligation(and(within(tank.
    location, roofSpace), has(tank,
    cover))))

Prediction with reversible IR:
if(is(pipedWaterSupplySystem, roofSpace
    )), then(obligation(has(
    pipedWaterSupplySystem, cover)))

Label:
if(and(has(pipedWaterSupplySystem,
    waterTank), within(waterTank.
    location, roofSpace))), then(
    obligation(has(waterTank, cover)))
```

**Fig. 11.** Example predictions from a model trained with entity extraction as lossy IR and a model trained only with the reversible IR. NZBC G12/AS1 Section 5.2.4 [66].

This reduction can be considered an essential step towards the lossy IR and self-reflection experiments where the generated IR was appended to the input text. The entity IR and self-reflection then brought the main F1 score gains.

On investigating the predictions for a semantic parser utilising the entity IR as exemplified in Fig. 11, one can notice the benefit of having a separate entity extraction step. The extracted entities *'tank'* and *'location'* enforced their correct usage in the generated LRML rule. Even though the prediction is not completely correct, it is closer to the ground truth. Furthermore, in a semi-automated process, one could simply change the extracted entity *'tank'* to *'water tank'* to fix this mistake, rather than having to rewrite the LRML rule directly.

Similar cases can be found throughout the test set. However, the opposite can also be true. If an unnecessary entity was extracted, this could lead to false expressions in the generated LRML rule. However, judging from the evaluation results, the lossy IR's benefit outweighed this disadvantage.

A more general contribution is the experimental outcome of Section 4.4, which indicates that having knowledge of previously predicted representations can positively influence the training. This could be due to having an example translation, which, in this case, belongs to the actual input clause. This phenomenon, including its adaptability to related tasks, should be further researched.

There are two primary threats to the validity of this study. First, using the DTS proposed in previous work allowed comparability between the results. Some of the results for the DTS did not reflect the conclusions drawn from the RTS. While this might be related to the DTS test set's smaller size and diversity, further examination will be required. Second, the LRML dataset is still a work in progress and shall be extended in future work. Especially important will be an independently created test set to strengthen the trustworthiness of the results and remediate the issues related to the DTS.

### 5.1. Dataset analysis

A dataset analysis was conducted to investigate the scalability of the semantic parsing approach (i.e., what types of regulations are included in the LRML dataset) and to what extent the RTS is a problem concerning sample overlap.

Fig. 12 provides insights into the constitution of the final LRML dataset and the implied complexity for semantic parsing. The samples were classified into prescriptive and constitutive statements and tagged when clauses were split or contained enumerations, lists, or multiple sentences. Lists were tagged whenever more than three elements, phrases, or sentences were listed. However, only one case for a list of sentences was in the dataset; all others were expressed as enumerations.

First of all, the dataset contains mostly prescriptive statements (n=658), but 60 constitutive statements, discussing the applicability of the AS and defining entities within the main content of the regulation document, are also included. Having 110 un-split enumerations, 66 clauses with multiple sentences, and 22 lists in the dataset significantly increases the semantic parsing difficulty and shows that the semantic parsing approach can cover these regulation elements. However, this coverage is only to the extent that the regulation elements can be translated into a single LRML rule. It remains open for future work to determine whether the semantic parsing model can formalise regulations into multiple LRML rules if required or if splitting all enumerations into separate statements for the formalisation process is better.

In contrast, 50 of these regulation elements (mostly enumerations) were split into 176 LRML rules to be formalised individually. This was strongly reduced from the original 92 clauses formalised as 287 LRML rules in Fuchs et al. [6]. This reduction comes not only from the data cleansing in Fuchs et al. [7] but the analysis in this paper was also more nuanced. The split clauses were not determined by the numbering but by manually classifying the similarly numbered clauses with actual overlaps. This more nuanced approach was necessary to evaluate the implications coming from the sample similarity.

The overlap differed widely between the various split enumerations, as shown in Fig. 13. While some of them had only a few words overlap, over half of the unique split clauses had at least a 2:1 ratio of shared words to distinct words, as shown in Fig. 14.

The implications of these overlaps were evaluated further for the RTS of the final dataset. Fig. 15 shows the distribution of the enumeration splits over the training, validation and test sets. A high transparency represents high ratios between the shared and unique word token counts.

Overall, there are 141 enumeration splits in the training set, 16 in the validation set, and 19 in the test set. However, only three samples in the test set and five samples in the validation set include unique split clauses with a high ratio of shared words to distinct words overlapping with the training set. While such cases are generally not preferred, the limited number is acceptable since these samples still evaluate if the model is overfitting on the training samples or if it successfully adapts the LRML rules for the new entities.

## 6. Conclusion

This paper demonstrated that reversible IRs could reduce the training time to almost a quarter and improve the semantic parsing quality by 2% F1 score. Using entity extraction as IR led to further improvements of up to 4.6% F1 score, and manually simplifying legal clauses enhanced the parsing quality by 18.4% to 84.8% F1 score.

The significance of these results is threefold. First, the increased quality of the predicted LRML rules is a step towards fully automated formalisation. While the scores are not yet perfect, the outputs are very promising. Currently, the evaluation is against a single possible translation. In future work, logically equivalent solutions should be

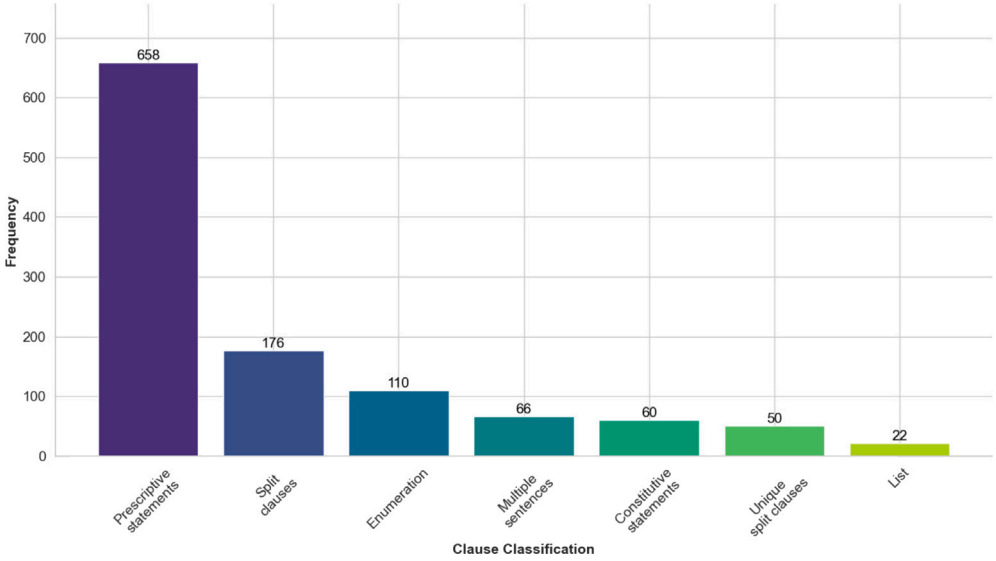



**Fig. 12.** Classification of the building code clauses.

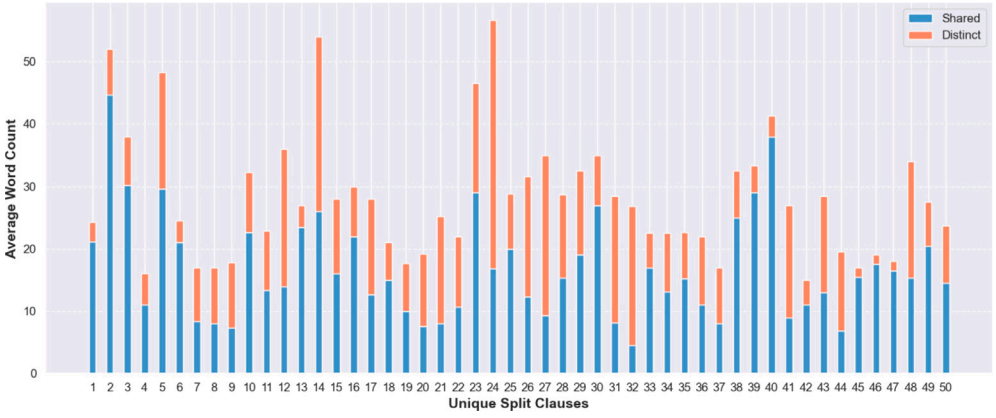



**Fig. 13.** Analysis of split enumerations. Comparing the average word count that is shared and distinct to each of the clauses.

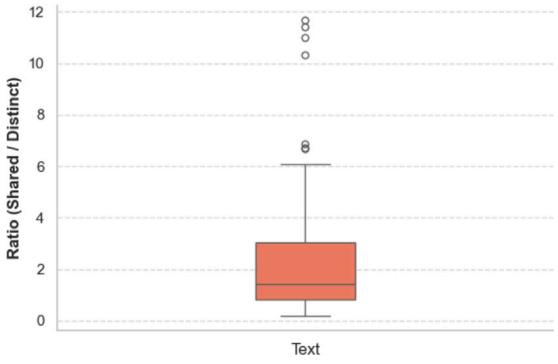



**Fig. 14.** Split enumeration shared to distinct word token ratio. The ratios were computed from the values in Fig. 13.

considered, and a more fine-grained assessment closer to the representation's end use should be added. Fuchs et al. [61] examined the suitability of LLMs for the semantic parsing task and their potential to increase the semantic parsing performance and provided a deeper qualitative analysis of the resulting LRML rules.

Second, shorter training and inference times allow potential collaboration between expert and machine. The semantic parser can generate initial translations, which experts can improve. Partial translations and auto-completion can limit the manual effort required for these improvements. Fuchs et al. [64] introduced a proof-of-concept of an LRML editor that integrates the semantic parser to provide full translations and auto-completion options.

Third, while entities and paraphrases as IRs bring improvements by themselves, integrated into a user interface, they would be reviewable and improve interpretability. IRs offer a novel form of expert collaboration by allowing experts to annotate entities to improve the generated formal representation and to paraphrase the legal clause instead of manipulating the formal representation directly.

Limitations with the current state of this research, to be addressed in future work, include:

1. The creation of the LRML dataset, as well as the cleansing process to improve the consistency of the representation, relied on a review process for quality control. While this methodology can result in biases and reviewing the representation cannot bring the same level of quality as having multiple experts translate the same clauses and ensure inter-annotator agreement, a motivation for using generative neural networks for this line of research was to be able to utilise existing formalisation efforts for training purposes.
   Instead of using parts of the existing data set for testing, the generalisation capability to an independently created test set

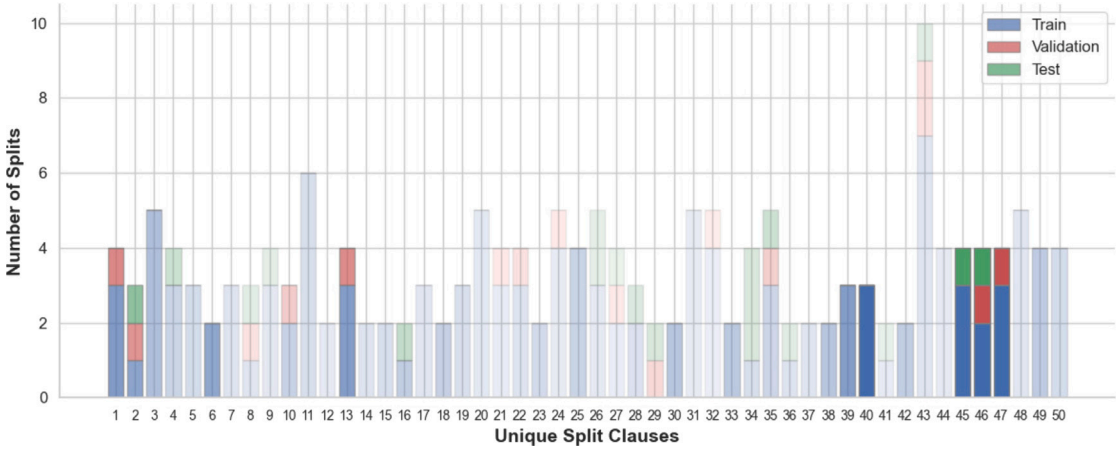



**Fig. 15.** Distribution of split enumerations over the training, validation, and test splits in the RTS. A high transparency represents high ratios between the shared and unique word token counts.

covering a range of different building regulations from different jurisdictions and covering different topics and complexities should be tested in future work.

2. The generalisation to unseen documents was significantly worse than to random test sets. On the one hand, this was expected since many of the terms and expressions are unknown to the semantic parser, which has to rely on its best guess. On the other hand, the behaviour of IRs for this document split was quite different than for random splits, which was ascribed to the differences in encoding styles and complexity. Testing the IRs with other document-based test splits could give more insights. While we expect the semantic parser to work similarly for unseen building codes and standards, fine-tuning on a limited set of exemplars from a new domain might still be necessary. However, retraining the model once new data is available can be easily automated.
3. Overall, there are cases where the semantic parser adds additional conditions, misses conditions, or swaps requirements with conditions, which would lead to false positives or false negatives in compliance checks. More focus should be put on a logic-based evaluation rather than relying on n-gram-based metrics, and a human-in-the-loop approach might be required to ensure a correct formalisation.
4. In many cases, there is more to the interpretation of building regulations than identifying the concepts and relationships between the concepts in regulations. Concepts can be abstract concepts which are not explicitly represented in the building design, such as an *'accessible path'*. While many of the concepts were resolved and implicit knowledge was identified, a detailed analysis of the alignment potential of the specified concepts needs to be conducted.
   It is expected that many of those concepts can be resolved by utilising the legal definitions. However, more problems can arise during the checking stage of the building. As an example, Ding et al. [18] showed that the check of accessible routes needs a lot of implicit knowledge formalised as additional rules to make the requirements checkable. Given Clause 7.1(a) in the Australian Standard AS1428.1, *"Accessible entrances shall be incorporated in an accessible path of travel"* [18], one must define that an *'entrance'* is an *'external door'*, a *'path'* consists of *'adjacent spaces'*, which are *'connected'* through a *'door'*, and for a path to be *'accessible'*, the *'minimum width'* of the path needs to be above a certain threshold (e.g., *'1200 mm'*) and the *'door'* needs to be *'accessible'*. Many of those concepts and relationships need to be made available by enriching the formal representation, the building design, and the ACC engine.

This research contributed advancements towards an ACC system by providing a method for establishing a correct and trustworthy legal foundation. The transformer-based end-to-end semantic parsing paradigm to support the formalisation of building regulations can make it more feasible to establish and maintain a certified formal representation of building regulations. Tools such as the LRML editor [64], in combination with the proposed intermediate representations, are envisioned to be used by regulators and domain experts to formalise existing regulations and draft new regulations as human and machine-interpretable rules in parallel. By continuously retraining and enhancing the semantic parser, manual interventions can be gradually reduced, ultimately leaving only the need to verify the generated formal representation.

Given the formalised legal requirements, the next step towards an ACC system will be to automatically generate CAPs, which includes the alignment to BIM and linking to the correct checking functionality. The automated generation of API calls by extracting the required parameters from LRML and IFC could improve the integration with external tools. Additionally, multi-modal classifiers will be explored to learn from previous compliance checking decisions.

## CRediT authorship contribution statement

**Stefan Fuchs:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Johannes Dimyadi:** Writing – review & editing, Supervision, Data curation. **Michael Witbrock:** Writing – review & editing, Supervision, Resources, Conceptualization. **Robert Amor:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

The source code and evaluation results are available at https://github.com/stefan-1992/intermediate-representations.

## Acknowledgements

This work was supported by the University of Canterbury's Quake Centre's Building Innovation Partnership (BIP) programme, which is jointly funded by industry and the Ministry of Business, Innovation and Employment (MBIE), New Zealand.

## References


[1] C. Eastman, J.-m. Lee, Y.-s. Jeong, J.K. Lee, Automatic rule-based checking of building designs, Autom. Constr. 18 (8) (2009) 1011–1033.

[2] W. Solihin, C. Eastman, Classification of rules for automated BIM rule checking development, Autom. Constr. 53 (2015) 69–82.

[3] S. Fuchs, R. Amor, Natural language processing for building code interpretation: A systematic literature review, in: Proc. of the Conference CIB W78, Vol. 2021, 2021, pp. 11–15.

[4] R. Zhang, N. El-Gohary, Natural language generation and deep learning for intelligent building codes, Adv. Eng. Inform. 52 (2022) 101557.

[5] Z. Zhang, N. Nisbet, L. Ma, T. Broyd, Capabilities of rule representations for automated compliance checking in healthcare buildings, Autom. Constr. 146 (2023) 104688.

[6] S. Fuchs, M. Witbrock, J. Dimyadi, R. Amor, Neural semantic parsing of building regulations for compliance checking, in: IOP Conference Series: Earth and Environmental Science, vol. 1101, 2022.

[7] S. Fuchs, J. Dimyadi, M. Witbrock, R. Amor, Training on digitised building regulations for automated rule extraction, in: EWork and EBusiness in Architecture, Engineering and Construction, ECPPM 2022, CRC Press, 2023.

[8] J. Dimyadi, G. Governatori, R. Amor, Evaluating legaldocml and legalruleml as a standard for sharing normative information in the aec/fm domain, in: Proceedings of the Joint Conference on Computing in Construction, Vol. 1, JC3, Heriot-Watt University, Edinburgh, UK. Heraklion, Greece, 2017, pp. 637–644.

[9] S. Fuchs, J. Dimyadi, M. Witbrock, R. Amor, Improving the semantic parsing of building regulations through intermediate representations, in: EG-ICE 2023 Workshop on Intelligent Computing in Engineering, Proceedings, 2023.

[10] R. Amor, J. Dimyadi, The promise of automated compliance checking, Dev. Built Environ. 5 (2021).

[11] J. Dimyadi, R. Amor, Automated building code compliance checking–where is it at, in: Proc. of CIB WBC, Vol. 6, No. 1, 2013.

[12] A.S. Ismail, K.N. Ali, N.A. Iahad, A review on BIM-based automated code compliance checking system, in: 2017 International Conference on Research and Innovation in Information Systems, Icriis, IEEE, 2017, pp. 1–6.

[13] T.H. Beach, J.L. Hippolyte, Y. Rezgui, Towards the adoption of automated regulatory compliance checking in the built environment, Autom. Constr. 118 (May) (2020) 103285, http://dx.doi.org/10.1016/j.autcon.2020.103285.

[14] M. Aydın, A review of BIM-based automated code compliance checking: A meta-analysis research, in: Automation and Control: Theories and Applications, 2022, p. 39, BoD–Books on Demand.

[15] W. Solihin, Lessons learned from experience of code-checking implementation in Singapore, in: BuildingSMART Conference, Singapore: BuildingSMART, 2004.

[16] L. Khemlani, CORENET e-PlanCheck: Singapore's automated code checking system, 2005, https://www.aecbytes.com/feature/2005/CORENETePlanCheck.html. (Retrieved 17 January 2024).

[17] W. Solihin, N. Shaikh, X. Rong, K. Poh, Beyond interoperatibility of building model: A case for code compliance checking, in: BPCAD Workshop, 2004.

[18] L. Ding, R. Drogemuller, M. Rosenman, D. Marchant, J. Gero, Automating code checking for building designs-DesignCheck, 2006.

[19] J. Wix, N. Nisbet, T. Liebich, Using constraints to validate and check building information models, in: ECPPM 2008 Conference, Sophia Antipolis, France, 2008, pp. 467–476.

[20] E. Hjelseth, N. Nisbet, Capturing normative constraints by use of the semantic mark-up RASE methodology, in: Proc. of CIB W78-W102 Conference, 2011.

[21] T.H. Beach, Y. Rezgui, H. Li, T. Kasim, A rule-based semantic approach for automated regulatory compliance in the construction sector, Expert Syst. Appl. 42 (12) (2015) 5219–5231.

[22] H. Lee, J.K. Lee, S. Park, I. Kim, Translating building legislation into a computer-executable format for evaluating building permit requirements, Autom. Constr. 71 (2016) 49–61.

[23] J. Song, J. Kim, J.K. Lee, NLP and deep learning-based analysis of building regulations to support automated rule checking system, in: ISARC. Proceedings of the International Symposium on Automation and Robotics in Construction, Vol. 35, IAARC Publications, 2018, pp. 1–7.

[24] J. Dimyadi, P. Pauwels, R. Amor, Modelling and accessing regulatory knowledge for computer-assisted compliance audit, J. Inf. Technol. Constr. (ITcon) 21 (21) (2016) 317–336.

[25] J. Zhang, N. El-Gohary, Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking, Autom. Constr. (Netherlands), Autom. Constr. 73 (2017) 45–57, http://dx.doi.org/10.1016/j.autcon.2016.08.027.

[26] J. Zhang, N. El-Gohary, Automated information transformation for automated regulatory compliance checking in construction, 29, (4) American Society of Civil Engineers (ASCE), Dept. of Civil and Environmental Engineering, Univ. of Illinois at Urbana-Champaign, 205 N. Mathews Ave., Urbana; IL; 61801, United States, 2015, http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000427,

[27] J. Zhang, N. El-Gohary, Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking, J. Comput. Civ. Eng. 30 (2) (2016).

[28] R. Zhang, N. El-Gohary, A machine-learning approach for semantic matching of building codes and building information models (BIMs) for supporting automated code checking, in: Recent Research in Sustainable Structures, Springer International Publishing, 2019, pp. 64–73, http://dx.doi.org/10.1007/978-3-030-34216-6_5.

[29] P. Zhou, N. El-Gohary, Ontology-based automated information extraction from building energy conservation codes, Autom. Constr. (Netherlands), Autom. Constr. 74 (2017) 103–117, http://dx.doi.org/10.1016/j.autcon.2016.09.004.

[30] P. Zhou, N. El-Gohary, Semantic information alignment of BIMs to computer-interpretable regulations using ontologies and deep learning, Adv. Eng. Inform. 48 (2021) 101239.

[31] Y.-C. Zhou, Z. Zheng, J.-R. Lin, X.-Z. Lu, Integrating NLP and context-free grammar for complex rule interpretation towards automated compliance checking, Comput. Ind. 142 (2022) 103746.

[32] Z. Zheng, X.-Z. Lu, K.-Y. Chen, Y.-C. Zhou, J.-R. Lin, Pretrained domain-specific language model for general information retrieval tasks in the AEC domain, 2022, arXiv preprint arXiv:2203.04729.

[33] R. Zhang, N. El-Gohary, Transformer-based approach for automated context-aware IFC-regulation semantic information alignment, Autom. Constr. 145 (2023) 104540.

[34] J. Wu, X. Xue, J. Zhang, Invariant signature, logic reasoning, and semantic natural language processing (NLP)-based automated building code compliance checking (i-SNACC) framework, J. Inf. Technol. Const. (ITcon) 28 (1) (2023).

[35] T. Bloch, R. Sacks, Comparing machine learning and rule-based inferencing for semantic enrichment of BIM models, Autom. Constr. 91 (2018) 256–272.

[36] S. Jiang, X. Feng, B. Zhang, J. Shi, Semantic enrichment for BIM: Enabling technologies and applications, Adv. Eng. Inform. 56 (2023) 101961, http://dx.doi.org/10.1016/j.aei.2023.101961, URL https://www.sciencedirect.com/science/article/pii/S1474034623000897.

[37] Z. Wang, R. Sacks, B. Ouyang, H. Ying, A. Borrmann, A framework for generic semantic enrichment of BIM models, J. Comput. Civ. Eng. 38 (1) (2024) 04023038.

[38] B. Li, C. Schultz, J. Dimyadi, R. Amor, Defeasible reasoning for automated building code compliance checking, in: ECPPM 2021–EWork and EBusiness in Architecture, Engineering and Construction, CRC Press, 2021, pp. 229–236.

[39] N.O. Nawari, A generalized adaptive framework (GAF) for automating code compliance checking, Buildings 9 (4) (2019) http://dx.doi.org/10.3390/buildings9040086, URL https://www.scopus.com/inward/record.uri?eid=2-s2.0-85065912070&doi=10.3390%2Fbuildings9040086&partnerID=40&md5=d8d65acdbb7297a9dda7dc1783695869.

[40] M. Palmirani, G. Governatori, T. Athan, H. Boley, A. Paschke, A. Wyner, LegalRuleML core specification version 1.0, 2017, OASIS Committee Specification Draft, 1.

[41] H.P. Lam, M. Hashmi, B. Scofield, Enabling reasoning with LegalRuleML, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 9718, Springer Verlag, 2016, pp. 241–257, http://dx.doi.org/10.1007/978-3-319-42019-6_16, URL http://link.springer.com/10.1007/978-3-319-42019-6_16.

[42] J. Dimyadi, S. Fernando, K. Davies, R. Amor, Computerising the New Zealand building code for automated compliance audit, in: 6th New Zealand Built Environment Research Symposium, Vol. 6, NZBERS2020, 2020, pp. 39–46.

[43] H. Boley, S. Tabet, G. Wagner, Design rationale for ruleml: A markup language for semantic web rules., in: SWWS, Vol. 1, 2001, pp. 381–401.

[44] Ministry of Business, Innovation and Employment, Acceptable Solutions and Verification Methods For New Zealand Building Code Clause G13 Foul water, second ed., 2019, amendment 8.

[45] L. Robaldo, C. Bartolini, M. Palmirani, A. Rossi, M. Martoni, G. Lenzini, Formalizing GDPR provisions in reified I/O logic: The DAPRECO knowledge base, J. Log. Lang. Inf. (2019) http://dx.doi.org/10.1007/s10849-019-09309-z.

[46] A.Z. Wyner, F. Gough, F. Levy, M. Lynch, A. Nazarenko, On annotation of the textual contents of scottish legal instruments., in: JURIX, 2017, pp. 101–106.

[47] G. Governatori, M. Hashmi, H.P. Lam, S. Villata, M. Palmirani, Semantic business process regulatory compliance checking using LegalRuleML, Lecture Notes in Comput. Sci. 10024 LNAI (690974) (2016) 746–761, http://dx.doi.org/10.1007/978-3-319-49004-5_48.

[48] P. Zhou, N. El-Gohary, Semantic information extraction of energy requirements from contract specifications: Dealing with complex extraction tasks, J. Comput. Civ. Eng. 36 (5) (2022) 04022025.

[49] Ministry of Business, Innovation and Employment, Acceptable Solutions and Verification Methods For New Zealand Building Code Clause D1 Access Routes, second ed., 2017, amendment 6.

[50] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (1) (2020) 5485–5551.

[51] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).
[52] L. Dong, M. Lapata, Coarse-to-fine decoding for neural semantic parsing, in: ACL 2018 - 56th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference (Long Papers), Vol. 1, 2018, pp. 731–742, http://dx.doi.org/10.18653/v1/p18-1068.
[53] J. Herzig, P. Shaw, M.-W. Chang, K. Guu, P. Pasupat, Y. Zhang, Unlocking compositional generalization in pre-trained models using intermediate representations, 2021, arXiv preprint arXiv:2104.07478.
[54] R. Zhang, N. El-Gohary, Hierarchical representation and deep learning–based method for automatically transforming textual building codes into semantic computable requirements, J. Comput. Civ. Eng. 36 (5) (2022) 04022022.
[55] G. Ferraro, H.P. Lam, S.C. Tosatto, F. Olivieri, M.B. Islam, N. van Beest, G. Governatori, Automatic extraction of legal norms: Evaluation of natural language processing tools, Lecture Notes in Comput. Sci. 12331 LNAI (October) (2020) 64–81, http://dx.doi.org/10.1007/978-3-030-58790-1_5.
[56] M.C. Mansouri, N.R. Ghlamallah, The impact of legal English idiosyncratic syntax on law readers: Between psycholinguistic dilemmas and preciseness, in: Al Athar Conference Proceedings, Vol. numéro 35SP, University of Ouargla, 2021, p. SP 2021, URL http://dspace.univ-ouargla.dz/jspui/handle/123456789/26320.
[57] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, Bleu: a method for automatic evaluation of machine translation, in: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 2002, pp. 311–318, http://dx.doi.org/10.3115/1073083.1073135, URL https://www.aclweb.org/anthology/P02-1040.
[58] R.J. Williams, D. Zipser, A learning algorithm for continually running fully recurrent neural networks, Neural Comput. 1 (2) (1989) 270–280.
[59] V. Aribandi, Y. Tay, T. Schuster, J. Rao, H.S. Zheng, S.V. Mehta, H. Zhuang, V.Q. Tran, D. Bahri, J. Ni, et al., Ext5: Towards extreme multi-task scaling for transfer learning, 2021, arXiv preprint arXiv:2111.10952.
[60] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F.L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, 2023, arXiv preprint arXiv:2303.08774.
[61] S. Fuchs, M. Witbrock, J. Dimyadi, R. Amor, Using large language models for the interpretation of building regulations, 2024, arXiv preprint arxiv:2407.21060.
[62] N. Shinn, B. Labash, A. Gopinath, Reflexion: an autonomous agent with dynamic memory and self-reflection, 2023, arXiv preprint arXiv:2303.11366.
[63] Ministry of Business, Innovation and Employment, Acceptable Solutions and Verification Methods For New Zealand Building Code Clause B1 Structure, first ed., 2018, amendment 17.
[64] S. Fuchs, J. Dimyadi, A.S. Ronee, R. Gupta, M. Witbrock, R. Amor, A Legal-RuleML editor with transformer-based autocompletion, in: EC3 Conference 2023, Vol. 4, European Council on Computing in Construction, 2023.
[65] S.M. Ilal, H.M. Günaydın, Computer representation of building codes for automated compliance checking, Autom. Constr. 82 (2017) 43–58.
[66] Ministry of Business, Innovation and Employment, Acceptable Solutions and Verification Methods For New Zealand Building Code Clause G12 Water Supplies, third ed., 2019, amendment 12.